DreamOS

Find your files by what they are about, not by what they are called

Final Year B.Tech Project | Final Monitoring, 14 October 2026

Zaid Ali Syed (2305139) Om Vyas (2305170) Krushna Kadam (2305165)
Under the guidance of Dr. Minakshi R. Rajput
Institute of Information and Communication Technology, MGM University

PART 1 - THE PROBLEM

Every computer has a folder like this

- final_final_v3.txt
- Untitled document.txt
- CV2.txt
- scan0007.txt
- backup_of_backup.txt
- new_doc_2026_03_11.txt
- notes.txt
- important_dont_delete.txt



Which one is the resume I sent to the company?

Computers find files by name. People remember what a file is about.

PART 1 - THE IDEA

What if you could just ask?

You type

“find my invoices”

“open the second one”

“what is related to my resume?”

DreamOS finds, opens and groups the files for you - on your own computer.

Like a librarian who has read every book in your house, and never tells anyone what is on your shelves.

No cloud. No account. No uploads.

PART 1 - OBJECTIVES

Six things we set out to do



Find by meaning

Search files by what they say, not what they are named



Understand requests

One box that works out whether you want to find, open, relate or organise



Organise safely

Tag and sort files, and undo every move



Show connections

Show which files belong together, and why



Remember the chat

Make follow-up questions like “open it” work



Keep data private

Never send files or questions to a server, and prove it

What already exists

Windows Search and Spotlight

Match the words in a query against the words in a file. A query has to share vocabulary with the file.

Everything

Extremely fast, but only looks at file names.

LSFS (ICLR 2025 paper)

Shows that searching files by meaning works at the operating-system level, but it is built for AI agents that call an API, not for a person typing.

The building blocks come from research on word and sentence meaning: Mikolov et al. (word vectors) and Reimers and Gurevych (Sentence-BERT). Plain version: each sentence becomes a point on a map, and similar sentences land close together.

PART 2 - RESEARCH GAPS

Where the existing work stops

1

Built for machines, not people

Meaning-based search is shown as an API for software agents. Nobody offers one simple box for a person that finds and acts.

2

Privacy is promised, not shown

Few systems show, in a way you can check, that no data leaves the computer.

3

Real limits are rarely reported

Short everyday queries and large folders are where meaning search breaks. We measure both and report what we find.

PART 3 - OUR SOLUTION

How DreamOS works, in four steps



1. Read

Each file is cut into pieces. A local AI model turns each piece into a point on the meaning map.



2. Index

The points and the file details are stored on your computer, in two local databases.



3. Understand

Your question is worked out: find, open, relate, organise, or suggest workspaces.

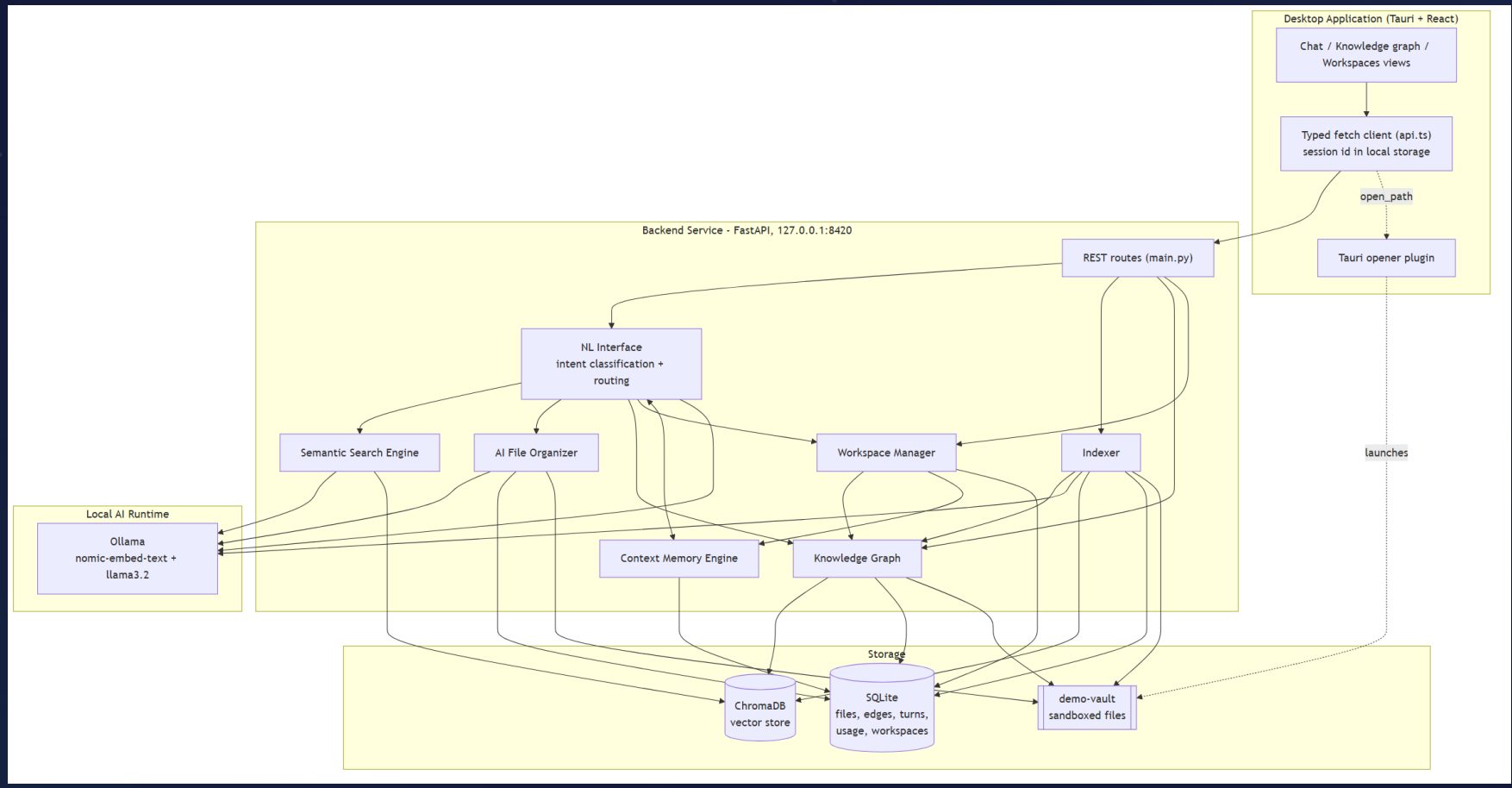


4. Answer

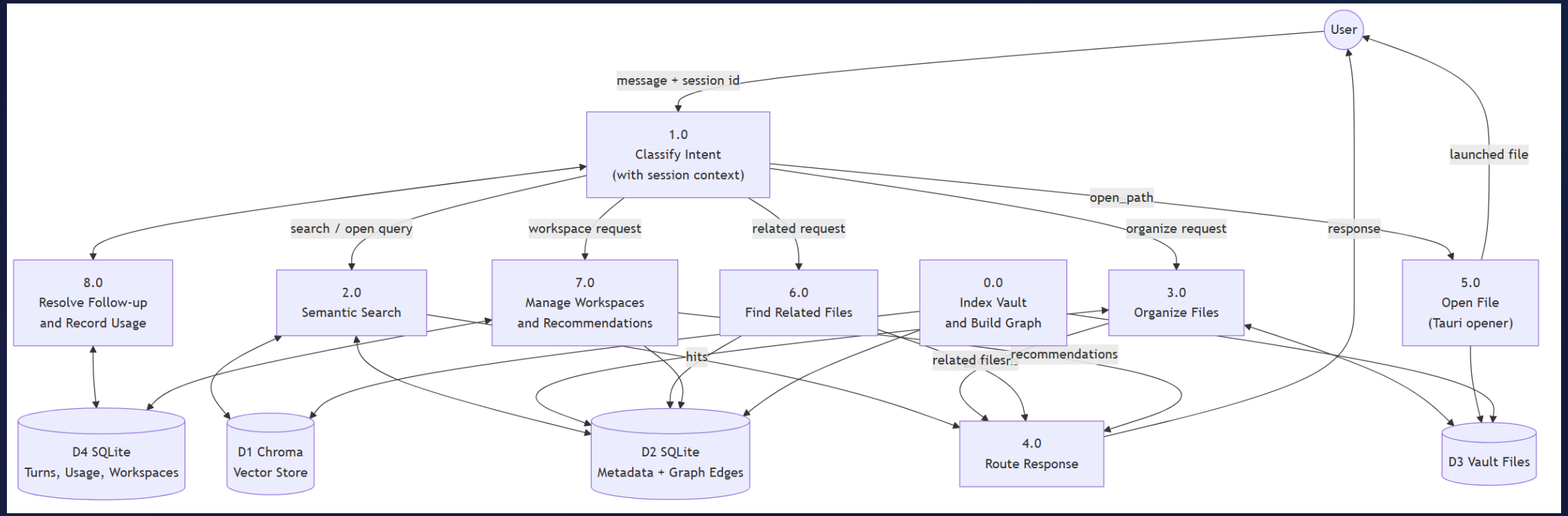
You get cards with the best files, the reason they matched, and buttons to act on them.

PART 3 - ARCHITECTURE

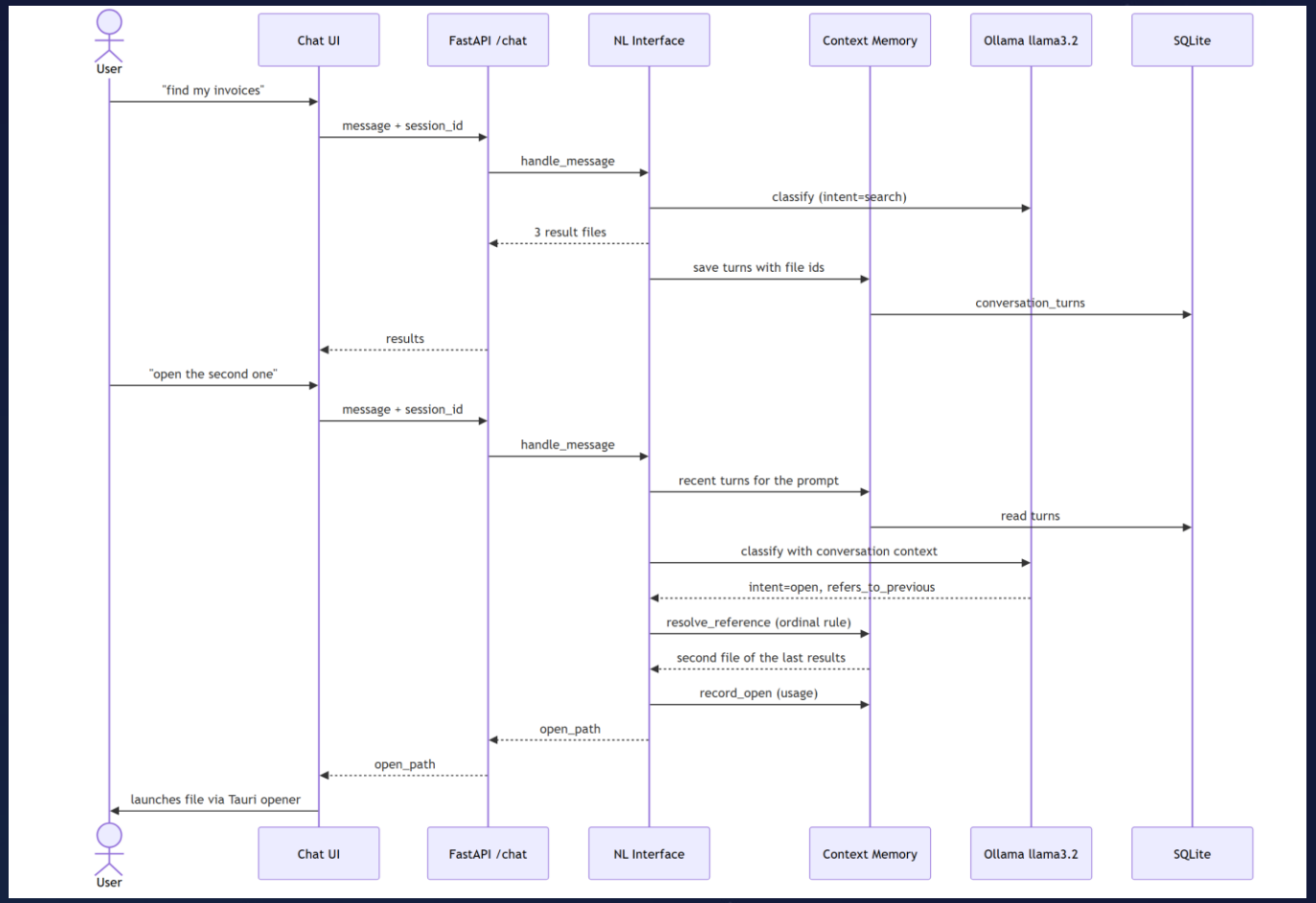
Everything runs on one computer



How a request flows through the system

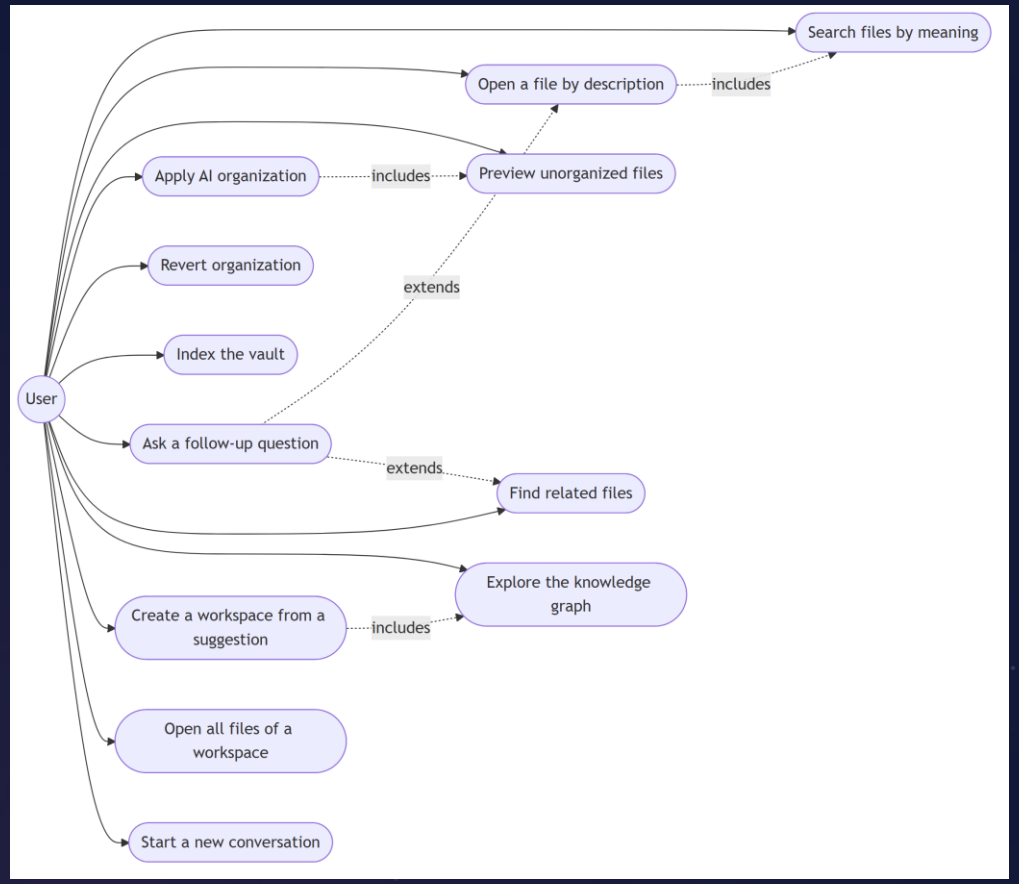


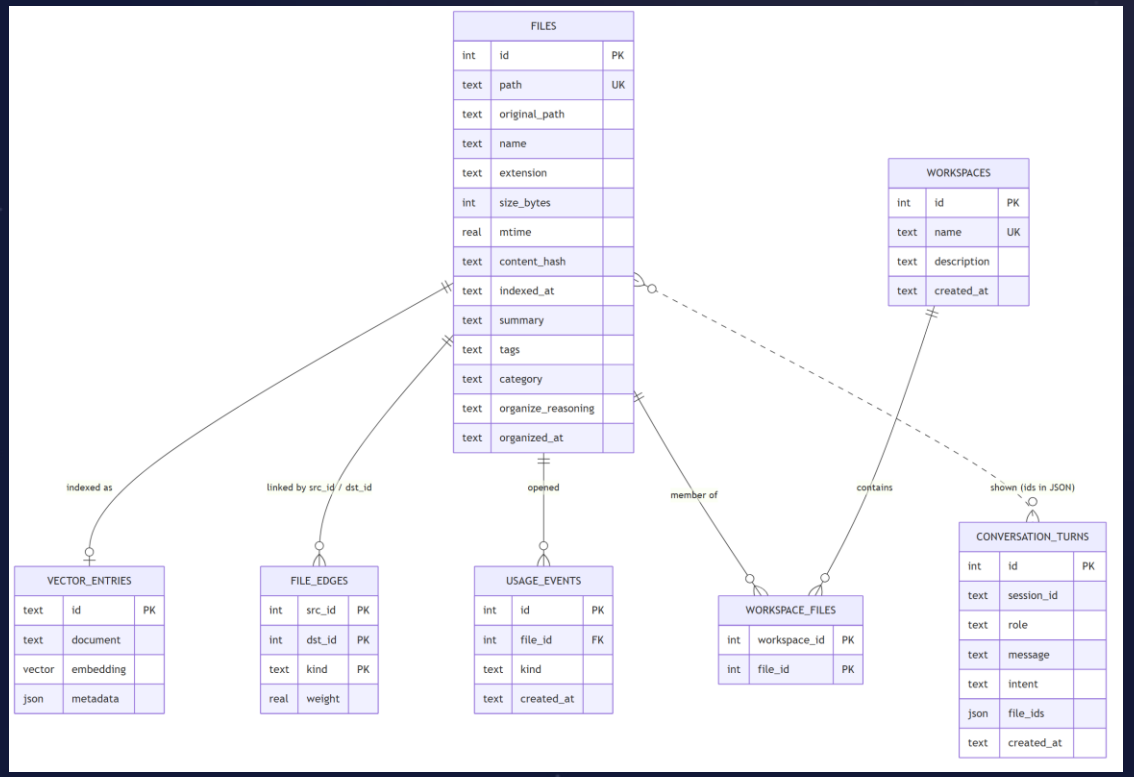
A follow-up question, step by step



- You ask for invoices; the answer is remembered with the files it showed.
- “open the second one” is worked out from that memory, not guessed.
- The app opens the file, and the choice is saved for next time.

What users can do, and how the data fits together





PART 3 - MODULES

Six modules, each with one job

01

Understanding

Works out what you want from a plain sentence

02

Search

Finds files close in meaning, and says when it is only a guess

03

Organiser

Suggests categories and tags, moves files, and can undo

04

Connections

Links files that are similar, that name each other, or share tags

05

Memory

Remembers the chat so follow-ups make sense

06

Workspaces

Suggests groups of related files and opens them together

KEY IDEA 1 - MEANING AS LOCATION

Similar files sit close together

- Every piece of text becomes a list of 768 numbers, a location on a very large map.
- Texts about the same thing land near each other, even with no words in common.
- Finding a file = finding the nearest points to your question.
- Plain comparison: a library shelved by subject, not by the author's surname.

A real example

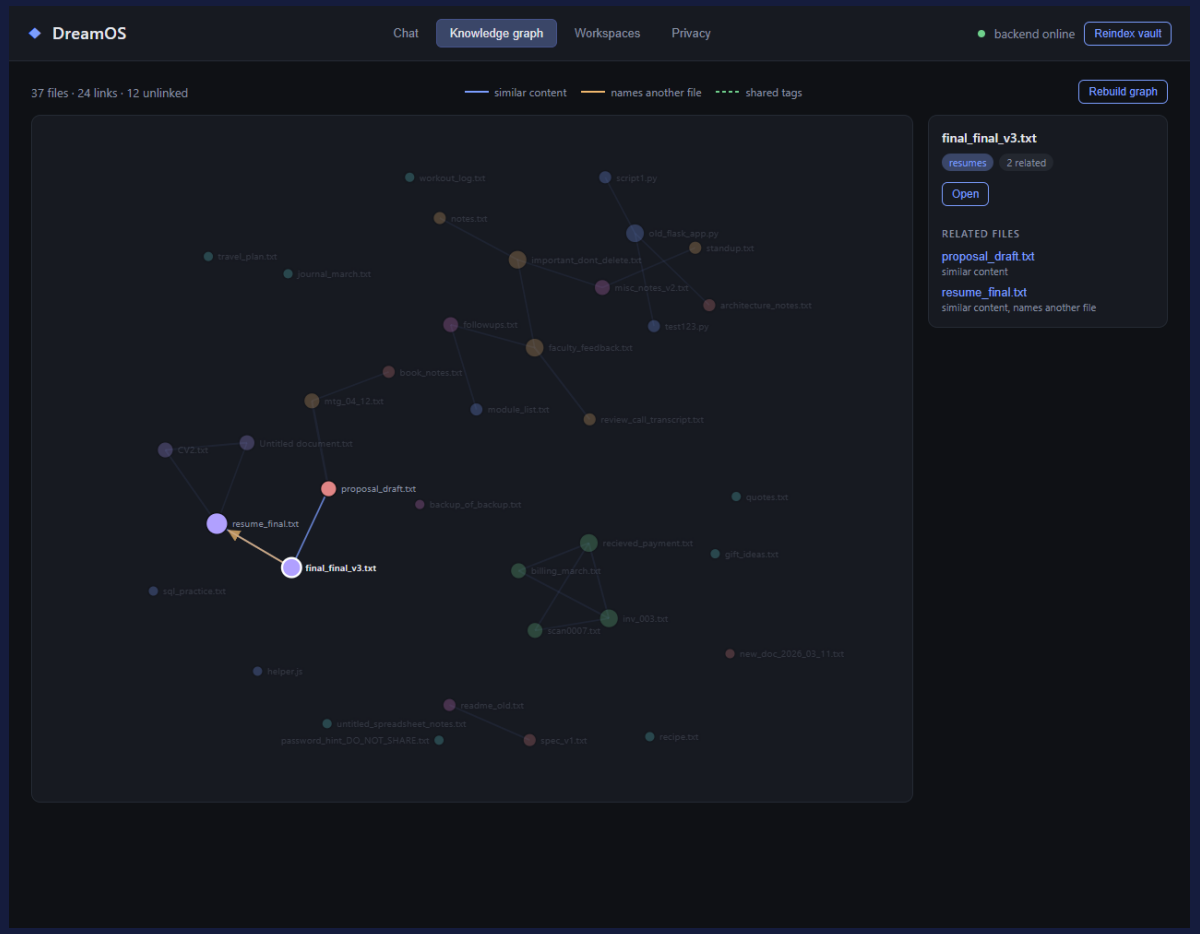
“find something about a meeting”

It returns the team notes, though the file names say nothing about meetings.

Honest note: very short queries are harder, and we measure that in the results section.

KEY IDEA 2 - CONNECTIONS

A map of how your files relate



- Three kinds of link: similar content, one file names another, shared tags.
- Each file keeps only its strongest links, so the picture stays readable.
- Click any file to see what it is connected to, and why.

KEY IDEA 3 - MEMORY

Follow-up questions just work

You

find my invoices

DreamOS

Found 2 file(s) matching ‘invoices’.

You

open the second one

DreamOS

Opening recieved_payment.txt...

Why this is hard: a computer has to know that “the second one” refers to the list it showed a moment ago. We decide that with rules first, and use the AI model only as a last resort.

KEY IDEA 4 - PRIVACY YOU CAN CHECK

Nothing leaves this computer



The app tells you where it sends requests. Right now the answer is: only to this computer.

Inside the app

- A Privacy tab lists every service DreamOS talks to
- It marks each one as “this computer only” or “another computer”
- If someone points it at a remote model, the warning turns red

Checking it yourself

Run `netstat -ano` in a terminal. The DreamOS service listens on 127.0.0.1 only, which means it cannot be reached from another computer.

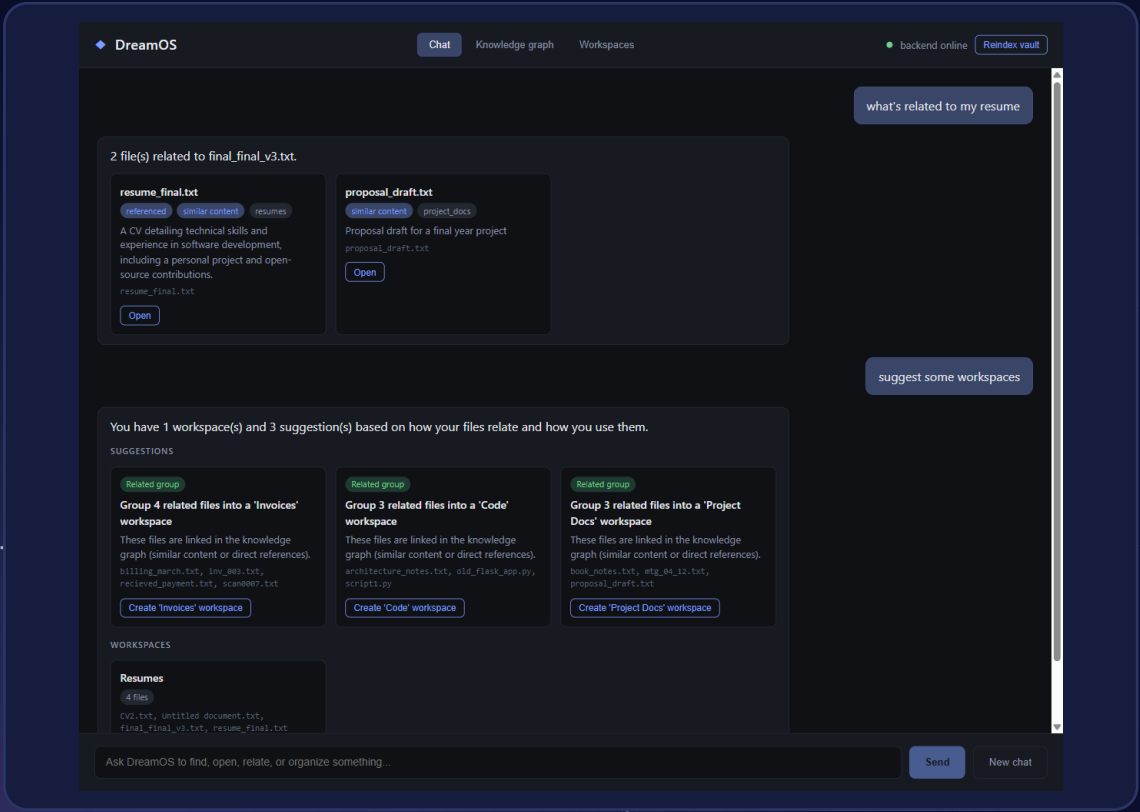
PART 3 - TECHNOLOGY

What it is built with

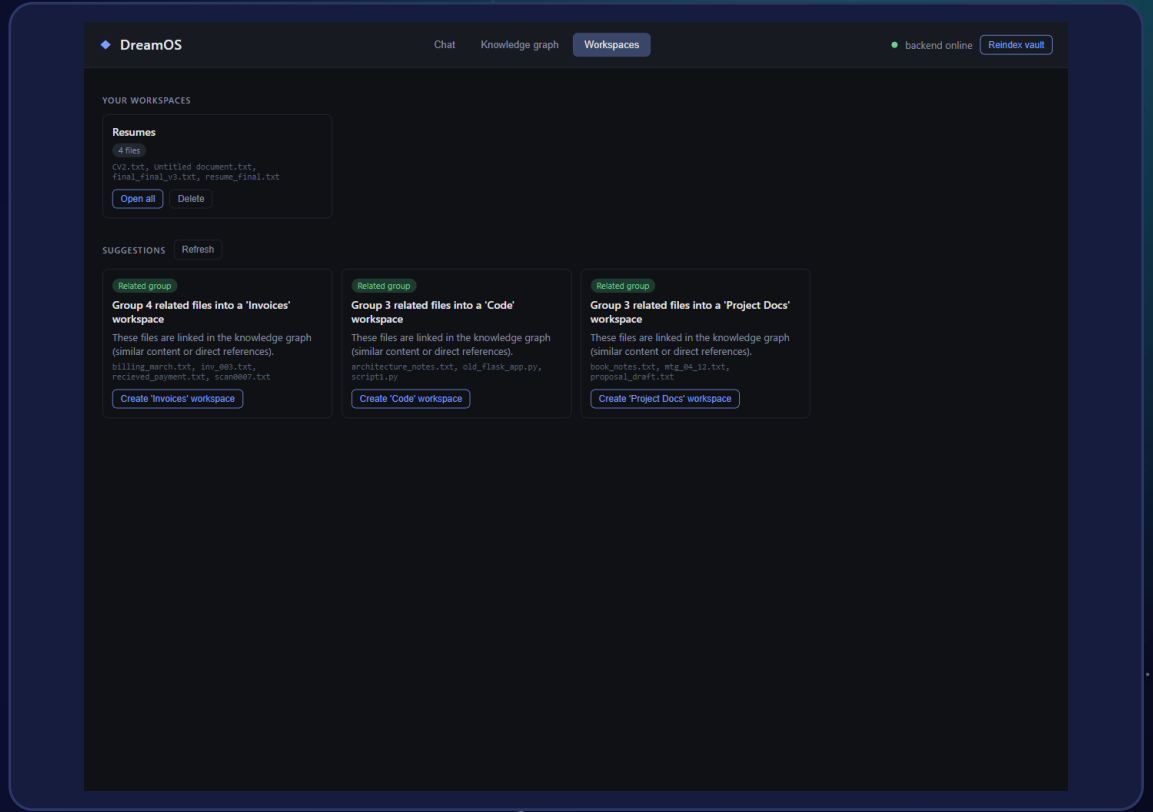
Desktop app	React, TypeScript, Tauri (a small native window)
Service	Python with FastAPI
AI models	Ollama running nomic-embed-text (meaning) and llama3.2 (understanding)
Storage	SQLite for details and history; ChromaDB for meaning vectors
Testing	pytest with 123 automated tests; GitHub Actions on every push

PART 3 - THE APP

What the user sees



Chat: ask in plain words; cards show why each file matched



Workspaces: suggested groups, one click to create and open

PART 4 - TESTING

How we checked it works

Small checks	One piece at a time: does the ranking, the memory rule and the reversal work?	*123 automated tests, run offline in under a minute*
Joined-up checks	Pieces together: index a folder, search it, organise it, undo it	*Run against the real AI models*
Real app	The desktop window itself, driven like a user would drive it	*Opened and checked inside the running app*

PART 4 - LARGE-SCALE TESTING

Could it cope with a real-sized folder?

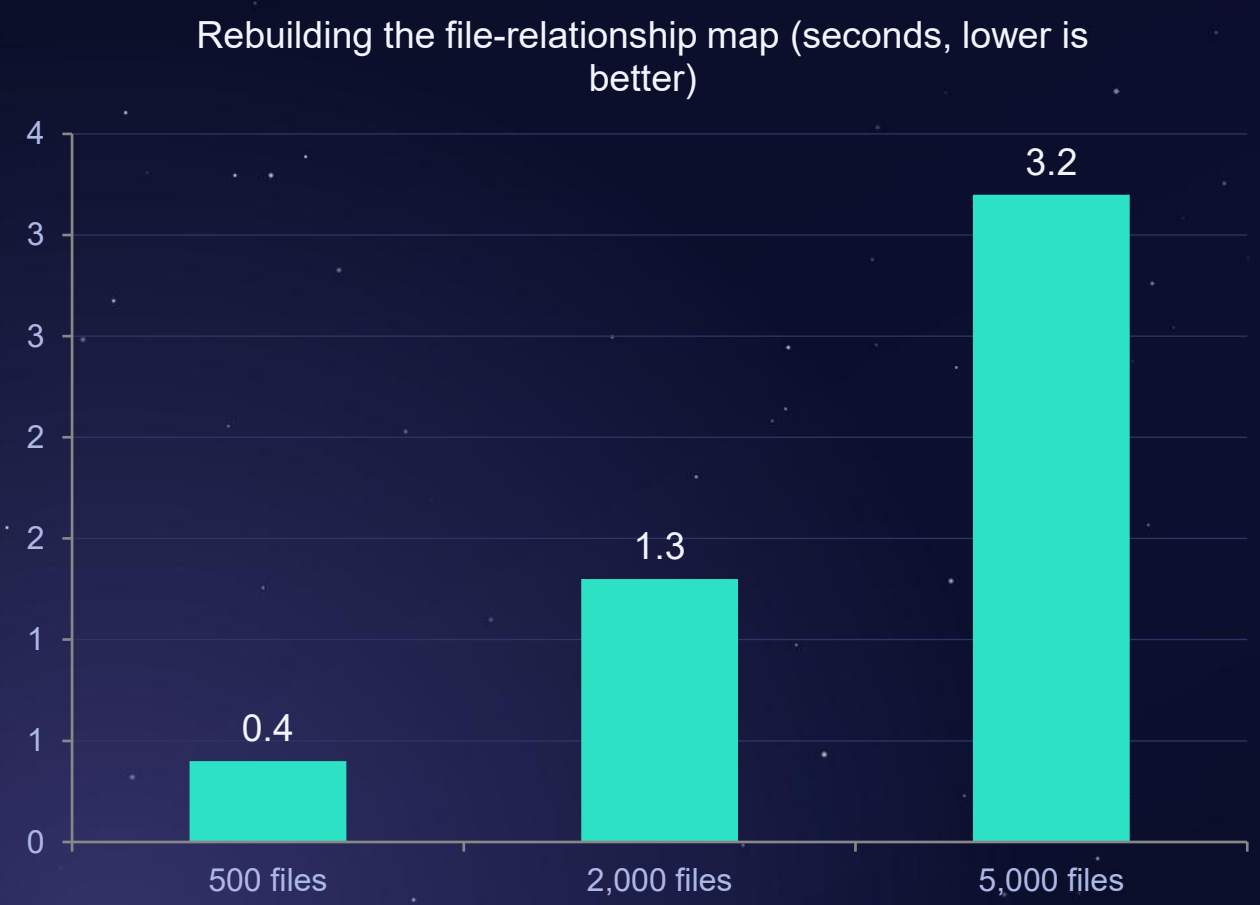
- We generated fake but realistic folders: 500, 2,000 and 5,000 files.
- Twelve topics: invoices, recipes, research notes, code, and more.
- Half of the files also carry a sentence from another topic, to make it harder.
- The real AI models did the work, on an ordinary laptop with no graphics card.
- Every number on the next slides comes from one script, which anyone can rerun.

The question we asked

If my folder has five thousand files, will I wait minutes for an answer?

PART 4 - RESULTS

Rebuilding the map of 5,000 files takes 3.2 seconds



276.3 s

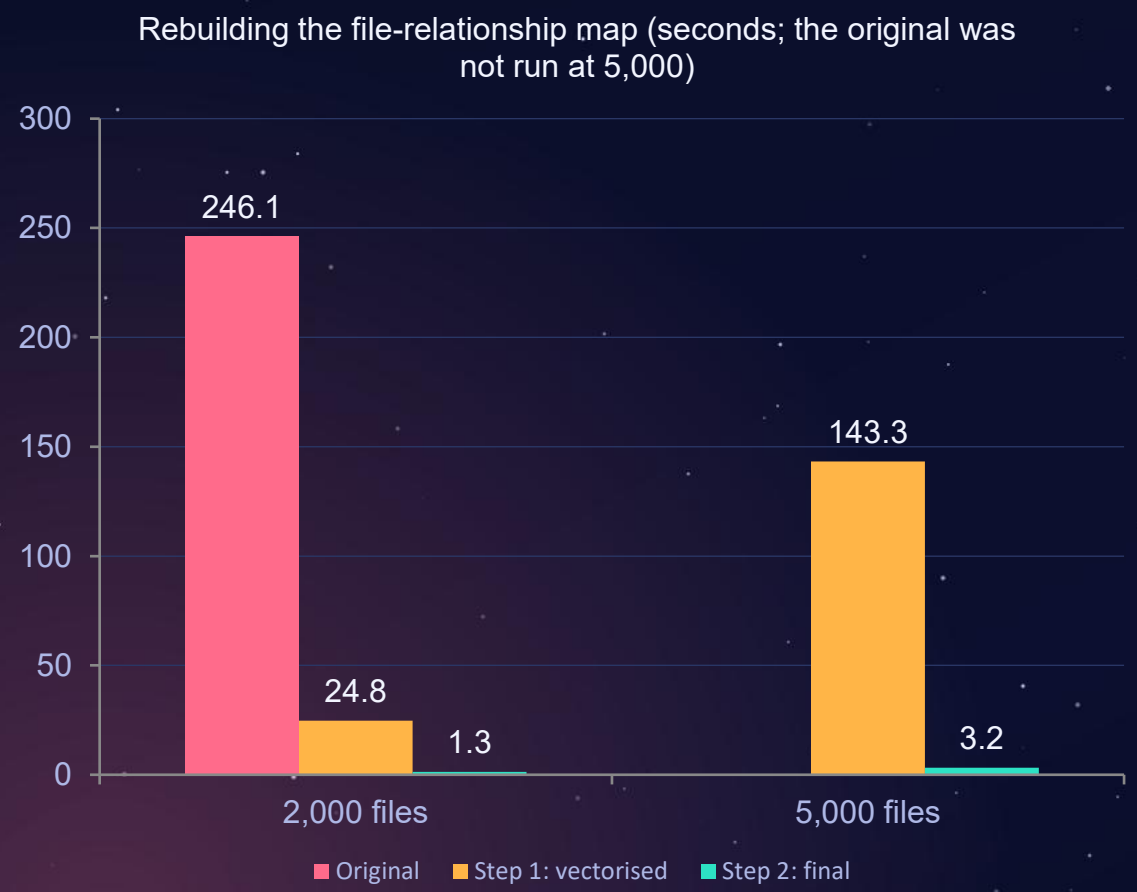
to index 5,000 files (about 18 files every second)

34.5 ms

median search time at 500 files; 204.7 ms at 5,000 files

PART 4 - RESULTS

Before and after fixing what the tests exposed



- Every embedding call waited about one second. The cause: the name "localhost" was tried over IPv6 first. Fix: use 127.0.0.1. Measured: about 1 s down to about 15 ms per call.
- The relationship map grew with the square of the file count. Step 1 vectorised it; then profiling showed 98% of the remaining time was one all-pairs check. Step 2 replaced it. Same links, 143.3 s down to 3.2 s at 5,000 files, with tests that the answers are unchanged.
- A follow-up about a new topic was answered with an old result. Fix: direct evidence now wins over the AI's guess, with a test that fails on the old behaviour.

PART 4 - RESULTS

Is the answer the right one?

1.00

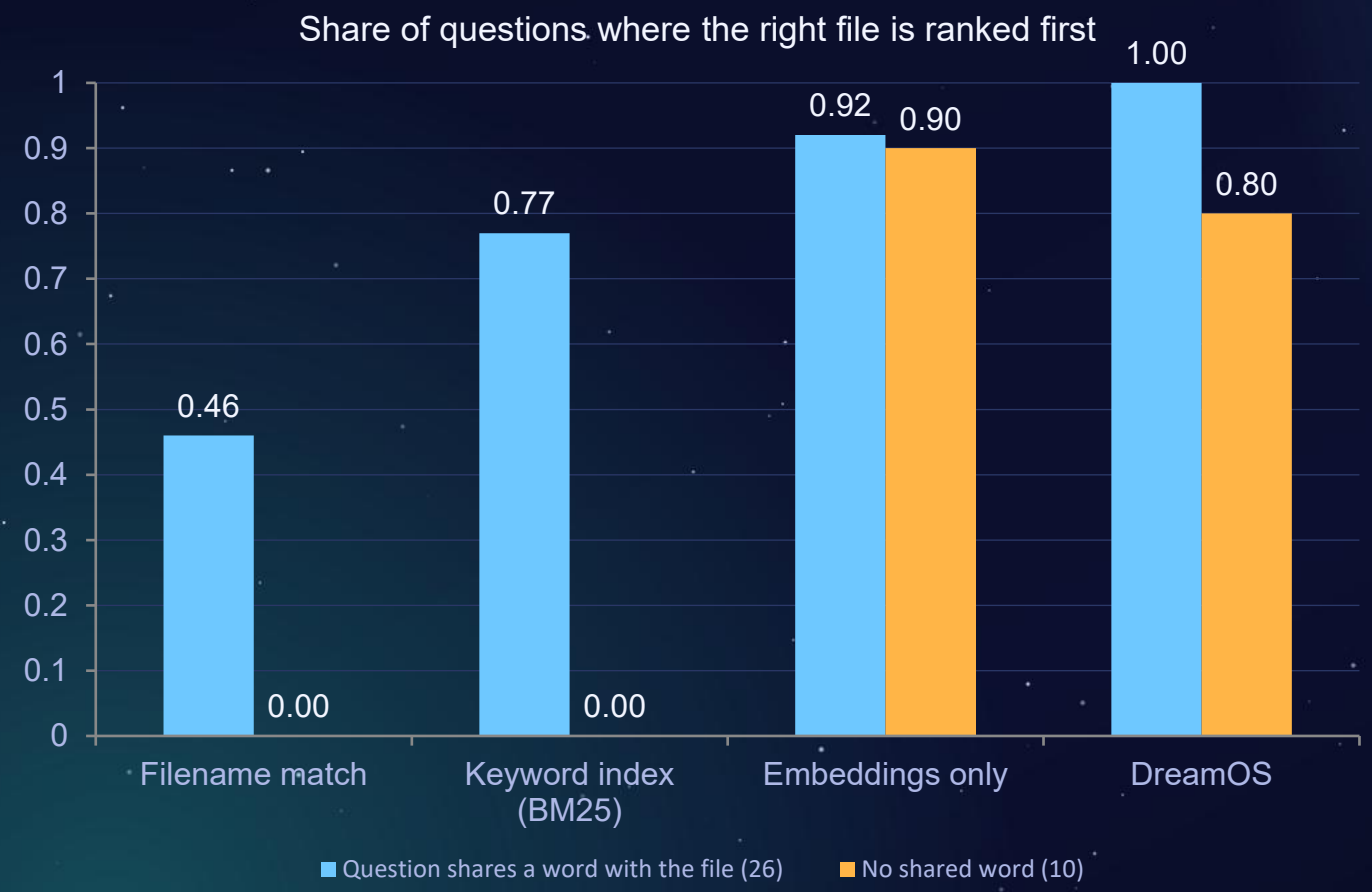
of the top 5 answers are on the right topic, on the 5,000-file set

What this does and does not show

- It shows that the topics separate cleanly in meaning at this size.
- It does not show quality on your real documents. Those are not in the test, on purpose.
- Test files were generated, so they are easier than real ones. We say so in the report.
- A harder test on real documents is the next step.

PART 4 - RESULTS

Does meaning beat keywords? We measured it



94%

first-place answers over 36 questions, against 56% for a keyword index and 33% for filename search

0.80 vs 0.00

when the question shares no word with the file, e.g. “present for my mother” finds the list that says “Mom's birthday”

PART 4 - RESULTS VALIDATION

How we know the results are right

Questions fixed before the run

36 questions and their correct files were written down first, so the test could not be bent to fit.

Honest split

A computer, not us, sorted questions into “shares a word” and “no shared word”.

Error bars shown

Small test set, so every score has a 95% interval; the report states the gaps that are not significant.

Fixes proved, not assumed

Each speed-up was checked against the original code on random data and produces identical results.

Failures listed

Two misses and one trade-off are reported, not hidden.

Privacy checkable

Anyone can run netstat and see the service only listens on this computer.

PART 5 - HONEST LIMITS

Where DreamOS is weaker

One-word questions

“meeting notes” can rank a file that only mentions a meeting above the real notes. We show such results as guesses, and never open a guess.

Borderline wording

A 3-billion-parameter model sometimes misreads the intent of a sentence. Measured on new phrasings: 88% correct.

Very large folders

Comparing every file with every other uses memory that grows with the square of the file count (about 200 MB at 5,000 files). Fine for thousands; tens of thousands would need an approximate method.

One computer, Windows only

Not designed for several users, phones, or syncing between machines.

PART 5 - CONCLUSION

What we delivered

- Six working modules, tested end to end
- A privacy check you can run yourself
- Tested at 5,000 files, with the weak spots reported
- Measured against keyword search: 94% vs 56% first-place answers
- Real defects found by testing and fixed, each with a test

Future scope

- Test on real documents, with consent
- Faster method for very large folders
- Scanned documents and images
- Installer that includes the AI models

PART 5 - TEAM

Individual contribution

Zaid Ali Syed

Roll No. 2305139

- System architecture and all six modules
- Desktop app (Tauri + React) and the privacy audit
- Scale benchmark, retrieval comparison and the performance fixes
- Research paper and the technical report content

Om Vyas

Roll No. 2305170

- Testing and evaluation
- Test cases and checking search quality
- Validation of results against expected answers

Krushna Kadam

Roll No. 2305165

- Documentation and presentation
- Report formatting and structure
- Slides and review of the final deliverables

Guide: Dr. Minakshi R. Rajput. All three members took part in design discussions, demo rehearsal and review.

QUESTIONS

Plain-English glossary

Embedding

A list of numbers that stands for the meaning of a piece of text

Vector database

A store that finds the nearest meaning-points quickly

Knowledge graph

A map of which files are linked, and why

Local model

An AI model that runs on this computer, with no internet

Cosine similarity

How close two meaning-points are, from 0 (unrelated) to 1 (the same)

Average linkage

A way to group files that avoids joining two groups by one weak link



Thank you

Questions and live demo

github.com/zaidwhy/dreamos-college-project (private, available to examiners)